AI Marketing Platform — System Architecture Design

Document Type: System Architecture Reference
Project: AI-Marketing-New (Enterprise SaaS)
Stack: Vanilla JS + Vite · ASP.NET Core 9 · PostgreSQL 16
Version: 1.0

Table of Contents

1. System Overview
2. High-Level Architecture
3. Container & Deployment Architecture
4. Frontend Architecture
5. Backend Architecture
6. Database Architecture
7. Authentication & Security Architecture
8. Multi-Tenancy Architecture
9. Role-Based Access Control (RBAC)
10. External Integration Architecture
11. Campaign & Creative Workflow Architecture
12. Approval Flow Architecture
13. API Architecture
14. Asset & File Storage Architecture
15. AI Integration Architecture
16. Service Layer Architecture
17. Data Flow Diagrams
18. Security Architecture
19. Scalability & Performance Design
20. Architecture Summary

---

1. System Overview

The **AI Marketing Platform** is a multi-tenant SaaS application that enables companies to plan, create, approve, deploy, and analyze digital advertising campaigns across multiple social media platforms — all powered by AI.

Core Capabilities

Capability	Description
Campaign Management	Full lifecycle from brief → approval → deploy → analytics
Multi-Platform Deployment	Facebook, Instagram, TikTok, Google Ads, YouTube
AI-Powered Content	Gemini 2.0 Flash generates ad copy, images, video scripts
Creative Asset Workflow	Platform-specific assets with aspect ratios, approval pipeline
Role-Based Approval	CMO → Expert → PPP → CMO multi-step approval system
Real-Time Analytics	Live performance metrics pulled from platform APIs
Multi-Tenancy	Full company isolation — each company sees only its own data

Brand Compliance	AI-powered brand guideline enforcement

User Roles

Role	Code	Description
Super Admin	`superadmin`	Platform-wide management, company provisioning
Admin	`admin`	Company-level user and role management
CMO	`cmo`	Strategic approvals, budget control, final sign-off
Marketing Expert	`expert`	Campaign creation, creative production, deployment
PPP (Performance)	`ppp`	Performance review, ad-wise budget entry, asset approval

---

2. High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          USERS (Browsers)                                 │
│         CMO │ Marketing Expert │ PPP │ Admin │ Super Admin                │
└──────────────────────────┬──────────────────────────────────────────────┘
                           │  HTTPS / HTTP (Port 80)
                           ▼
┌──────────────────────────────────────────────────────────────────────────┐
│                       NGINX REVERSE PROXY                                 │
│                         (Docker Container)                                │
│                                                                           │
│   /          → Serve Vanilla JS SPA (Static Files)                       │
│   /api/*     → Proxy → Backend :5243                                      │
│   /assets/*  → Proxy → Backend :5243 (static asset files)                 │
│   /library/* → Proxy → Backend :5243 (asset library files)                │
└──────────────────────────┬──────────────────────────────────────────────┘
           │                              │
           ▼                              ▼
┌──────────────────────┐       ┌──────────────────────────────────────┐
│  FRONTEND (SPA)      │       │          BACKEND API SERVER          │
│  Vanilla JS + Vite   │       │       ASP.NET Core 9 Minimal API     │
│  Tailwind CSS 4.x    │       │            (Port 5243)               │
│                      │       │                                      │
│  36 Screens          │◄──────►│  95+ REST Endpoints                  │
│  Role-based routing  │  JWT  │  EF Core 9 ORM                       │
│  Single Page App     │       │  BCrypt Auth                         │
│  No frameworks       │       │  JWT HS256 Token                     │
└──────────────────────┘       └──────────────┬───────────────────────┘
                                              │
                ┌─────────────────────────────┼─────────────────────────┐
                │                             │                         │
                ▼                             ▼                         ▼
┌──────────────────────┐   ┌──────────────────────────┐  ┌──────────────────────┐
│   POSTGRESQL 16      │   │   EXTERNAL AD PLATFORMS  │  │   GOOGLE GEMINI      │
```

```
│   (Port 5432)       │   │                          │  │   AI API          │
│                     │   │  • Facebook Graph v19    │  │                   │
│  29 Tables          │   │  • TikTok Business v1.3  │  │  Gemini 2.0 Flash │
│  Multi-tenant       │   │  • Google Ads API        │  │  Text Generation  │
│  Company isolation  │   │  • YouTube Data API      │  │  Image Generation │
│  EF Core auto-schema│   │  • Instagram (via FB)    │  │  Brand Analysis   │
└─────────────────────┘   └──────────────────────────┘  └───────────────────┘
```

---

3. Container & Deployment Architecture

Docker Compose Stack

```
docker-compose.yml
│
├── Service: db (postgres:16-alpine)
│     Port:    5432 (internal)
│     Volume:  pgdata (persistent PostgreSQL data)
│     Health:  pg_isready check every 5s
│
├── Service: backend (custom Dockerfile)
│     Port:    5243:5243
│     Volumes: assets:/app/Assets
│              library:/app/Assets Library
│     Env:     ConnectionStrings, JWT config, Gemini API Key
│     Depends: db (waits for healthy status)
│
└── Service: frontend (Dockerfile.frontend → nginx)
      Port:    80:80
      Build:   Vite build → static files → nginx serving
      Depends: backend
      Config:  nginx.conf (SPA routing + API proxy)
```

Docker Build Pipeline

```
┌──────────────────────────────────────────────────────────┐
│                  BUILD STAGE (multi-stage)                 │
│                                                           │
│  Frontend Build:                Backend Build:            │
│  ┌──────────────────┐           ┌──────────────────────┐  │
│  │ node:20-alpine   │           │ mcr.microsoft.com/   │  │
│  │ npm install      │           │ dotnet/sdk:9.0       │  │
│  │ vite build       │           │ dotnet publish       │  │
│  │ → /dist          │           │ → /app/publish       │  │
│  └──────────┬───────┘           └──────────┬───────────┘  │
│             │                              │              │
│             ▼                              ▼              │
│  ┌──────────────────┐           ┌──────────────────────┐  │
│  │ nginx:alpine     │           │ dotnet/aspnet:9.0    │  │
│  │ Copy /dist       │           │ Copy /app/publish    │  │
│  │ Copy nginx.conf  │           │ ENTRYPOINT dotnet    │  │
```

```
│  │ Expose 80         │           │ Expose 5243         │   │
│  └──────────────────┘           └────────────────────┘   │
└──────────────────────────────────────────────────────────┘
```

Network Flow

```
Internet
    │
    ▼
[Port 80] ──► NGINX Container
                  │
                  ├── Static JS/CSS/HTML ──► Browser
                  │
                  └── /api/* ──► [Port 5243] ──► Backend Container
                                                      │
                                                      └── [Port 5432] ──► DB Container
```

---

4. Frontend Architecture

Technology Stack

Component	Technology	Version
Language	Vanilla JavaScript (ES Modules)	ES2022+
Build Tool	Vite	7.2.4
CSS Framework	Tailwind CSS	4.x (Vite plugin)
PostCSS	autoprefixer	10.4.24
Architecture	Single Page Application (SPA)	—

Application Structure

```
frontend/
├── index.html              ← Entry HTML, mounts #app div
├── vite.config.js          ← Vite + Tailwind plugin config
├── package.json            ← Dependencies
└── src/
    ├── main.js             ← Core SPA router (3200+ lines)
    │     ├── renderScreen()     ← Master routing switch (36 cases)
    │     ├── Auth functions     ← login(), logout(), getToken()
    │     ├── All screen renders ← renderDashboard(), renderCampaigns()...
    │     └── API call helpers   ← apiFetch(), apiPost()...
    │
    ├── facebookDeploy.js   ← Facebook/Instagram deployment logic
    ├── tiktokDeploy.js     ← TikTok deployment logic
    ├── googleAdsDeploy.js  ← Google Ads deployment logic
    ├── youtubeAdsDeploy.js ← YouTube Ads deployment logic
```

```
    ├── style.css           ← Global styles + Tailwind imports
    └── counter.js          ← Utility helpers
```

SPA Routing Architecture

```
┌──────────────────────────────────────────────────────────────┐
│                        main.js Router                         │
│                                                               │
│  User Login                                                   │
│      │                                                        │
│      ▼                                                        │
│  JWT Token stored in localStorage                             │
│      │                                                        │
│      ▼                                                        │
│  renderScreen(screenId)   ◄── All navigation goes here        │
│      │                                                        │
│      ├── 'dashboard'           → renderDashboard()            │
│      ├── 'campaigns'           → renderCampaigns()            │
│      ├── 'creative-hub'        → renderCreativeHub()          │
│      ├── 'approvals'           → renderApprovalsScreen()      │
│      ├── 'ppp-queue'           → renderPPPQueue()             │
│      ├── 'cmo-queue'           → renderCMOQueue()             │
│      ├── 'budget-matrix'       → renderBudgetMatrix()         │
│      ├── 'analytics'           → renderAnalytics()            │
│      ├── 'ad-performance'      → renderRealAdPerformance()    │
│      ├── 'deploy-selection'    → renderEnhancedDeploySelection()│
│      ├── 'ab-testing'          → renderABTesting()            │
│      ├── 'user-management'     → renderUserManagement()       │
│      ├── 'role-management'     → renderRoleManagement()       │
│      ├── 'company-management'  → renderCompanyManagement()    │
│      └── ... (36 total screens)                               │
└──────────────────────────────────────────────────────────────┘
```

Screen Distribution by Role

```
Super Admin  (5 screens):  Company Mgmt, User Mgmt, System Logs,
                           Platform Config, Super Dashboard

Admin        (4 screens):  User Mgmt, Role Mgmt, Screen Permissions,
                           Company Profile

CMO          (8 screens):  Dashboard, Budget Matrix, CMO Queue,
                           Campaign Overview, Analytics, Ad Performance,
                           Final Approvals, Reports

Expert      (12 screens):  Dashboard, Campaigns, Ad Sets, Ads,
                           Creative Hub, Asset Library, Templates,
                           Deploy Selection, A/B Testing,
                           Brand Compliance, Notifications, AI Tools

PPP          (7 screens):  Dashboard, PPP Queue, Ad Review,
```

```
                              Budget Entry, Performance, Analytics,
                              Approval History
```

---

5. Backend Architecture

Technology Stack

Component	Technology	Version
Runtime	ASP.NET Core	9.0
Language	C#	13
API Style	Minimal API	—
ORM	Entity Framework Core	9
Database Provider	Npgsql (EF Core)	9.x
Auth	JWT Bearer + BCrypt	HS256
AI	Google Gemini SDK	2.0 Flash
Facebook	Graph API Client	v19.0
TikTok	Business API	v1.3
Google/YouTube	Google Ads API SDK	Latest

Backend File Structure

```
backend/
├── Program.cs              ← ALL 95+ endpoints (Minimal API, ~984 lines)
├── Models.cs               ← ALL 29 EF Core model classes (56.3 KB)
├── AppDbContext.cs         ← DbContext: DbSets, indexes, constraints
├── appsettings.json        ← Connection string, JWT config
├── appsettings.example.json← Template (no secrets)
├── backend.csproj          ← Project file + NuGet packages
│
├── Services/
│   ├── BrandComplianceService.cs    ← AI brand guideline checker
│   ├── DeploymentOrchestrator.cs   ← Multi-platform deploy coordinator
│   ├── FacebookAdsService.cs       ← Facebook Graph API calls
│   ├── GoogleAdsService.cs         ← Google Ads API calls
│   ├── MetricsFetchService.cs      ← Pull analytics from all platforms
│   └── YouTubeAdsService.cs        ← YouTube Data API calls
│
├── Middleware/             ← Custom request pipeline middleware
├── Assets/                 ← Generated ad image files (Docker volume)
├── Assets Library/         ← Reusable asset library (Docker volume)
└── Scripts/                ← DB seed/utility scripts
```

Minimal API Request Pipeline

```
HTTP Request
    │
    ▼
┌──────────────────────────────────────────────────┐
│              ASP.NET Core Pipeline                  │
│                                                    │
│  1. HTTPS Redirection                              │
│  2. Static Files (Assets, Library)                 │
│  3. CORS Policy (AllowAll in dev)                  │
│  4. JWT Authentication Middleware                  │
│  5. Authorization Middleware                       │
│  6. Custom Middleware (Logging, Error handling)    │
│  7. Endpoint Routing                               │
│       │                                            │
│       ├── /api/auth/*       → No auth required     │
│       ├── /api/admin/*      → [Authorize] required  │
│       ├── /api/campaigns/*  → [Authorize] required  │
│       └── ... (95+ routes)                        │
└──────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────┐
│            Business Logic Layer                     │
│  (inline lambda handlers in Program.cs)            │
│                                                    │
│  → EF Core DbContext queries                       │
│  → Service layer calls (Facebook, Google, AI)      │
│  → JWT claims extraction (company_id, role, userId) │
│  → Response DTO mapping                            │
└──────────────────────────────────────────────────┘
    │
    ▼
┌──────────────────────────────────────────────────┐
│              Data Access Layer                      │
│  EF Core 9 → Npgsql → PostgreSQL 16               │
└──────────────────────────────────────────────────┘
```

---

6. Database Architecture

Overview

- **Database:** PostgreSQL 16 (Alpine)
- **ORM:** Entity Framework Core 9 with `EnsureCreated` (auto-schema, no migration files)
- **Total Tables:** 29
- **Multi-tenancy:** `company_id` column on every table (NULL = Super Admin scope)

Table Groupings

```
┌──────────────────────────────────────────────────────────────────────┐
│                    DATABASE SCHEMA (29 Tables)                        │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  ┌── MULTI-TENANCY ──────────────────────────────────────────┐       │
│  │  companies          ← Root tenant table                   │       │
│  └───────────────────────────────────────────────────────────┘       │
│                                                                      │
│  ┌── AUTH & RBAC ────────────────────────────────────────────┐       │
│  │  users              ← All users (bcrypt password, role ref)       │
│  │  roles              ← Role definitions per company        │       │
│  │  screens            ← Screen/page registry                │       │
│  │  role_screens       ← Role ↔ Screen permissions (composite PK)    │
│  │  user_screens       ← User-level overrides (composite PK) │       │
│  └───────────────────────────────────────────────────────────┘       │
│                                                                      │
│  ┌── CAMPAIGN CORE ──────────────────────────────────────────┐       │
│  │  campaigns          ← Campaign header                     │       │
│  │  ad_sets            ← Ad sets under campaigns             │       │
│  │  ads                ← Individual ads under ad sets        │       │
│  │  budgets            ← Budget allocations                  │       │
│  └───────────────────────────────────────────────────────────┘       │
│                                                                      │
│  ┌── WORKFLOW & APPROVAL ────────────────────────────────────┐       │
│  │  ppp_queue          ← PPP review queue (new)              │       │
│  │  cmo_queue          ← CMO final approval queue            │       │
│  │  approvals          ← Approval decision log               │       │
│  │  notifications      ← In-app notification system          │       │
│  └───────────────────────────────────────────────────────────┘       │
│                                                                      │
│  ┌── ANALYTICS & PERFORMANCE ────────────────────────────────┐       │
│  │  ad_performance     ← Real metrics from platforms         │       │
│  │  ab_tests           ← A/B test configurations             │       │
│  │  ab_test_results    ← A/B test result data                │       │
│  └───────────────────────────────────────────────────────────┘       │
│                                                                      │
│  ┌── CREATIVE & ASSETS ──────────────────────────────────────┐       │
│  │  creative_briefs    ← Creative brief documents            │       │
│  │  asset_library      ← Reusable asset metadata             │       │
│  │  templates          ← Reusable ad templates               │       │
│  └───────────────────────────────────────────────────────────┘       │
│                                                                      │
│  ┌── PLATFORM & DEPLOYMENT ──────────────────────────────────┐       │
│  │  ad_accounts        ← Connected platform ad accounts      │       │
│  │  deployments        ← Deployment history log              │       │
│  │  platforms          ← Platform registry (FB, TikTok, etc.)        │
│  └───────────────────────────────────────────────────────────┘       │
│                                                                      │
│  ┌── BRAND & CONTENT ────────────────────────────────────────┐       │
│  │  brand_guidelines   ← Brand rules per company             │       │
│  │  content_calendar   ← Scheduled content plan              │       │
```

```
│  └──────────────────────────────────────────────────────────┘  │
│                                                                 │
│  ┌── SYSTEM ──────────────────────────────────────────────────┐  │
│  │  audit_logs         ← System-wide audit trail              │  │
│  │  system_settings    ← Platform configuration               │  │
│  └────────────────────────────────────────────────────────────┘  │
│                                                                 │
└─────────────────────────────────────────────────────────────────┘
```

Key Relationships

```
companies (1)
    │
    ├──(n) users
    │         └──(1) roles
    │                 └──(n) role_screens ──(n) screens
    │                         └── user_screens (override)
    │
    ├──(n) campaigns
    │         └──(n) ad_sets
    │                  └──(n) ads
    │                           ├──(1) ppp_queue
    │                           ├──(1) cmo_queue
    │                           ├──(n) approvals
    │                           ├──(n) ad_performance
    │                           └──(n) deployments
    │
    ├──(n) brand_guidelines
    ├──(n) asset_library
    ├──(n) templates
    ├──(n) ad_accounts
    └──(n) notifications
```

Index Strategy

Index Type	Columns	Purpose
Unique	`users.email`	Prevent duplicate accounts
Unique	`companies.slug`	URL-safe unique tenant identifier
Unique	`screens.screen_key`	Prevent duplicate screen registration
Composite PK	`role_screens(role_id, screen_id)`	RBAC permission mapping
Composite PK	`user_screens(user_id, screen_id)`	Per-user screen override
Index	`ads.campaign_id`	Fast ad lookup by campaign
Index	`ad_performance.ad_id, fetched_at`	Time-series performance queries
Index	`notifications.user_id, is_read`	Unread notification count

---

7. Authentication & Security Architecture

JWT Token Flow

```
┌─────────┐          ┌─────────────────┐          ┌──────────┐
│ Client  │          │ Backend (Auth)  │          │ Database │
└────┬────┘          └────────┬────────┘          └────┬─────┘
     │                        │                        │
     │ POST /api/auth/login   │                        │
     │ { email, password }    │                        │
     │ ──────────────────────►│                        │
     │                        │ SELECT user WHERE email │
     │                        │ ──────────────────────►│
     │                        │◄──────────────────────│
     │                        │ BCrypt.Verify(password) │
     │                        │                        │
     │                        │ Generate JWT HS256:    │
     │                        │ Claims: {              │
     │                        │   sub: userId,         │
     │                        │   role: "expert",      │
     │                        │   company_id: "123",   │
     │                        │   exp: now + 8h        │
     │                        │ }                      │
     │                        │                        │
     │ { token, user }        │                        │
     │◄──────────────────────│                        │
     │                        │                        │
     │ Store in localStorage  │                        │
     │                        │                        │
     │ GET /api/campaigns     │                        │
     │ Authorization: Bearer <token>                   │
     │ ──────────────────────►│                        │
     │                        │ Validate JWT           │
     │                        │ Extract company_id     │
     │                        │ Apply tenant filter    │
     │                        │ ──────────────────────►│
     │                        │ WHERE company_id = X   │
     │ { campaigns[] }        │◄──────────────────────│
     │◄──────────────────────│                        │
```

JWT Configuration

```
Algorithm:  HS256
Expiry:     8 hours
Key:        Environment variable (Jwt__Key)
Issuer:     MarketingAIBackend
Audience:   MarketingAIFrontend

Claims Embedded:
  - sub          → userId
```

```
  - role         → user role name
  - company_id   → tenant isolation key
  - name         → display name
  - email        → user email
```

Password Security

```
Storage:   BCrypt hash (never plain text)
Rounds:    Default BCrypt cost factor (12)
Verify:    BCrypt.Verify(inputPassword, storedHash)
Reset:     Via admin user management screen
```

---

8. Multi-Tenancy Architecture

Tenant Isolation Model

```
┌──────────────────────────────────────────────────────────────┐
│                    MULTI-TENANCY DESIGN                       │
│                                                               │
│  Approach: Shared Database, Shared Schema                     │
│  Isolation: company_id on every table                        │
│                                                               │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  JWT Token (every authenticated request)              │   │
│  │  ┌──────────────────────────────────────────────┐    │   │
│  │  │  { company_id: "42", role: "expert", ... }      │    │   │
│  │  └──────────────────────────────────────────────┘    │   │
│  │              │                                        │   │
│  │              ▼ Automatically applied to ALL queries      │   │
│  │  SELECT * FROM campaigns WHERE company_id = 42        │   │
│  │  SELECT * FROM users     WHERE company_id = 42        │   │
│  │  SELECT * FROM ads       WHERE company_id = 42        │   │
│  └──────────────────────────────────────────────────────┘   │
│                                                               │
│  Super Admin Exception:                                       │
│  company_id = NULL in JWT → can see ALL companies            │
│  Uses /api/superadmin/* endpoints                            │
└──────────────────────────────────────────────────────────────┘
```

Company Provisioning Flow

```
Super Admin
    │
    ├── POST /api/superadmin/companies  → Create company
    ├── POST /api/superadmin/users      → Create Admin user for company
    │
Admin (per company)
    │
    ├── POST /api/admin/users    → Add Expert / CMO / PPP users
```

```
    ├── POST /api/admin/roles     → Define custom roles
    └── PUT  /api/admin/role-screens → Assign screens to roles
```

---

9. Role-Based Access Control (RBAC)

RBAC Architecture

```
┌──────────────────────────────────────────────────────────────────┐
│                      RBAC System Design                          │
│                                                                  │
│  Level 1: Role-Based (default)                                   │
│  ┌──────────────┐    ┌────────────────┐    ┌──────────────────────┐    │
│  │   roles      │──→│ role_screens  │←──│       screens        │    │
│  │ (cmo, ppp)   │    │ (role+screen)  │    │ (36 screen registry) │    │
│  └──────────────┘    └────────────────┘    └──────────────────────┘    │
│                                                                  │
│  Level 2: User Override (granular)                               │
│  ┌──────────────┐    ┌────────────────┐    ┌──────────────────────┐    │
│  │   users      │──→│ user_screens  │←──│       screens        │    │
│  │(individual)  │    │(user+screen)   │    │ (override per user)  │    │
│  └──────────────┘    └────────────────┘    └──────────────────────┘    │
│                                                                  │
│  Resolution Order:                                               │
│  1. Check user_screens (user-specific override)                  │
│  2. If no override → check role_screens (role default)           │
│  3. If not in either → screen is BLOCKED                         │
└──────────────────────────────────────────────────────────────────┘
```

Screen Permission Matrix (Key Screens)

Screen	Super Admin	Admin	CMO	Expert	PPP
Company Management	✅	❌	❌	❌	❌
User Management	✅	✅	❌	❌	❌
Role Management	✅	✅	❌	❌	❌
Budget Matrix	❌	❌	✅	✅	❌
CMO Queue	❌	❌	✅	❌	❌
Creative Hub	❌	❌	❌	✅	❌
Campaign Create	❌	❌	❌	✅	❌
Deploy	❌	❌	❌	✅	❌
PPP Queue	❌	❌	❌	❌	✅
Ad Performance	❌	❌	✅	✅	✅
Analytics	❌	❌	✅	✅	✅

10. External Integration Architecture

Platform Integration Map

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        EXTERNAL INTEGRATIONS                             │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │  SOCIAL MEDIA / AD PLATFORMS                                     │  │
│  │                                                                  │  │
│  │  ┌──────────────────┐    ┌──────────────────┐                  │  │
│  │  │  Facebook Graph  │    │  TikTok Business │                  │  │
│  │  │  API v19.0       │    │  API v1.3        │                  │  │
│  │  │                  │    │                  │                  │  │
│  │  │  • Create Ads    │    │  • Create Ads    │                  │  │
│  │  │  • Set Budgets   │    │  • Set Budgets   │                  │  │
│  │  │  • Pull Metrics  │    │  • Pull Metrics  │                  │  │
│  │  │  • Instagram too │    │  • Video Ads     │                  │  │
│  │  └──────────────────┘    └──────────────────┘                  │  │
│  │                                                                  │  │
│  │  ┌──────────────────┐    ┌──────────────────┐                  │  │
│  │  │  Google Ads API  │    │  YouTube Data    │                  │  │
│  │  │  (latest)        │    │  API             │                  │  │
│  │  │                  │    │                  │                  │  │
│  │  │  • Search Ads    │    │  • Video Ads     │                  │  │
│  │  │  • Display Ads   │    │  • Pull Views    │                  │  │
│  │  │  • Pull Metrics  │    │  • Pull Metrics  │                  │  │
│  │  └──────────────────┘    └──────────────────┘                  │  │
│  └──────────────────────────────────────────────────────────────────┘  │
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │  AI SERVICES                                                     │  │
│  │                                                                  │  │
│  │  ┌──────────────────────────────────────────────────────┐      │  │
│  │  │  Google Gemini 2.0 Flash                             │      │  │
│  │  │                                                      │      │  │
│  │  │  • Generate ad copy (text)                           │      │  │
│  │  │  • Generate image descriptions                       │      │  │
│  │  │  • Brand compliance checking                         │      │  │
│  │  │  • Campaign performance insights                     │      │  │
│  │  │  • A/B test analysis                                 │      │  │
│  │  │  • Creative brief generation                         │      │  │
│  │  └──────────────────────────────────────────────────────┘      │  │
│  └──────────────────────────────────────────────────────────────────┘  │
└──────────────────────────────────────────────────────────────────────────┘
```

Ad Account Connection Flow

```
Admin/Expert
    │
```

```
    ├── Add Ad Account → POST /api/ad-accounts
    │     { platform: "facebook", account_id, access_token, company_id }
    │
    ├── Stored in: ad_accounts table (encrypted access_token)
    │
    └── Used for:
          ├── Deploy  → FacebookAdsService.CreateAd(account_id, ad)
          ├── Metrics → MetricsFetchService.FetchFacebookMetrics()
          └── Budget  → FacebookAdsService.UpdateBudget(account_id, budget)
```

---

11. Campaign & Creative Workflow Architecture

Campaign Lifecycle State Machine

```
                        ┌──────────────────────────────────┐
                        │         Campaign States          │
                        └──────────────────────────────────┘


  ┌──────────┐    ┌──────────┐    ┌──────────┐    ┌──────────┐
  │  DRAFT   │──►│ PENDING  │──►│ APPROVED │──►│  LIVE    │
  │          │    │ APPROVAL │    │          │    │          │
  └──────────┘    └──────────┘    └──────────┘    └──────────┘
       │               │              │               │
       │               │ (Rejected)   │               │
       │               ▼              │               ▼
       │          ┌──────────┐        │          ┌──────────┐
       └──────────►│ REJECTED │        │          │ COMPLETE │
                  └──────────┘        │          └──────────┘
                                      │               │
                                      │               ▼
                                      │          ┌──────────┐
                                      └──────────►│  PAUSED  │
                                                  └──────────┘
```

Creative Asset Generation Flow

```
CMO (Budget & Matrix Screen)
    │
    └── Defines: allowed platforms for this campaign
              (Facebook, Instagram, TikTok, Google, YouTube)
                        │
                        ▼
Marketing Expert (Creative Hub Screen)
    │
    ├── Selects platforms (from CMO-allowed list)
    ├── Defines per-platform:
    │     ├── Aspect Ratios (16:9, 9:16, 1:1, 4:5...)
    │     ├── Number of Images
    │     ├── Number of Videos
    │     └── Video Duration (seconds)
```

```
    │
    └── Triggers AI Asset Generation
                        │
                        ▼
        ┌──────────────────────────────────────────┐
        │         AI Generation Engine             │
        │                                          │
        │  For each platform × aspect ratio:       │
        │  ┌──────────────────────────────────┐    │
        │  │  POST /api/ai/generate-creative  │    │
        │  │  → Gemini 2.0 Flash              │    │
        │  │  → Returns: text, image URL, metadata │  │
        │  └──────────────────────────────────┘    │
        │                                          │
        │  Stored as: assets (file + asset_library) │
        └──────────────────────────────────────────┘
                        │
                        ▼
        Each Asset Record Contains:
        ┌──────────────────────────┐
        │  • Content/Text          │
        │  • Platform              │
        │  • Aspect Ratio          │
        │  • File URL              │
        │  • Duration (video only) │
        │  • Status: pending_review │
        └──────────────────────────┘
```

---

12. Approval Flow Architecture

Multi-Step Approval Pipeline

```
┌──────────────────────────────────────────────────────────────────────┐
│                    APPROVAL PIPELINE ARCHITECTURE                    │
└──────────────────────────────────────────────────────────────────────┘


STEP 1: Expert Creates & Submits
────────────────────────────────

  Marketing Expert
      │
      ├── Creates Campaign + Ad Sets + Ads
      ├── Generates creative assets in Creative Hub
      └── POST /api/ppp-queue  →  Status: "pending_ppp"
                 │
                 ▼
          ppp_queue table
          { ad_id, status: "pending", submitted_by: expert_id }
                 │
                 ▼  (Notification sent to PPP user)


STEP 2: PPP Reviews & Budgets
```

```
────────────────────────────
  PPP User
      │
      ├── GET /api/ppp-queue          → View all pending ads
      ├── Reviews creative assets
      ├── Enters ad-wise budget       → PUT /api/ppp-queue/:id/budget
      └── Approves/Rejects           → PUT /api/ppp-queue/:id/approve
                                           or /reject
            │
            ▼ (On approve)
         ppp_queue.status = "approved_ppp"
         cmo_queue created           → Status: "pending_cmo"
            │
            ▼  (Notification sent to CMO)

STEP 3: CMO Final Approval
──────────────────────────
  CMO
      │
      ├── GET /api/cmo-queue          → View PPP-approved ads
      ├── Reviews assets + budgets
      └── Final Approve/Reject       → PUT /api/cmo-queue/:id/approve
                                           or /reject
            │
            ▼ (On CMO approve)
         campaign.status = "approved"
         ads.status = "ready_to_deploy"
            │
            ▼  (Notification sent to Expert)

STEP 4: Expert Deploys
──────────────────────
  Marketing Expert
      │
      ├── GET /api/deploy-selection   → View approved ads
      └── POST /api/deploy            → Push to selected platform
            │
            ▼
         DeploymentOrchestrator.cs
         │
         ├── platform == "facebook"  → FacebookAdsService.Deploy()
         ├── platform == "tiktok"    → TikTokDeploy logic
         ├── platform == "google"    → GoogleAdsService.Deploy()
         └── platform == "youtube"  → YouTubeAdsService.Deploy()
            │
            ▼
         deployments table
         { ad_id, platform, external_ad_id, deployed_at, status }
            │
            ▼
         ads.status = "live"
```

Approval Status Flow Summary

```
Draft → Submitted → Pending PPP → PPP Approved → Pending CMO
                                ↓                          ↓
                          PPP Rejected              CMO Approved → Live
                                                          ↓
                                                    CMO Rejected → Back to Expert
```

---

13. API Architecture

Endpoint Organization

```
┌──────────────────────────────────────────────────────────────────┐
│                    REST API DESIGN                                 │
│                                                                    │
│  Base URL: /api/                                                   │
│  Style:    ASP.NET Core Minimal API                                │
│  Auth:     JWT Bearer Token on all non-auth routes                 │
│  Format:   JSON (request/response)                                 │
│                                                                    │
│  ┌──────────────────────────────────────────────────────────┐    │
│  │ DOMAIN                 │ ENDPOINTS │ AUTH REQUIRED          │    │
│  ├────────────────────────┼───────────┼────────────────────────┤    │
│  │ /api/auth/*            │    4      │ ❌ (public)            │    │
│  │ /api/superadmin/*      │    8      │ ✅ SuperAdmin role     │    │
│  │ /api/admin/*           │    6      │ ✅ Admin role          │    │
│  │ /api/rbac/*            │    8      │ ✅ Admin role          │    │
│  │ /api/campaigns/*       │   10      │ ✅ Any auth            │    │
│  │ /api/ad-sets/*         │    6      │ ✅ Any auth            │    │
│  │ /api/ads/*             │    8      │ ✅ Any auth            │    │
│  │ /api/ppp-queue/*       │    6      │ ✅ PPP + CMO           │    │
│  │ /api/cmo-queue/*       │    5      │ ✅ CMO role            │    │
│  │ /api/assets/*          │    5      │ ✅ Any auth            │    │
│  │ /api/deploy/*          │    4      │ ✅ Expert role         │    │
│  │ /api/ai/*              │    6      │ ✅ Any auth            │    │
│  │ /api/analytics/*       │    5      │ ✅ Any auth            │    │
│  │ /api/ab-tests/*        │    4      │ ✅ Any auth            │    │
│  │ /api/budgets/*         │    4      │ ✅ CMO + PPP           │    │
│  │ /api/templates/*       │    4      │ ✅ Any auth            │    │
│  │ /api/notifications/*   │    3      │ ✅ Any auth            │    │
│  │ /api/ad-accounts/*     │    3      │ ✅ Admin role          │    │
│  └──────────────────────────────────────────────────────────┘    │
│                                                  Total: 95+        │
└──────────────────────────────────────────────────────────────────┘
```

API Request/Response Pattern

```
// Standard Request
POST /api/campaigns
```

```
Authorization: Bearer <JWT>
Content-Type: application/json

{
  "name": "Summer Campaign 2025",
  "platform": "facebook",
  "objective": "conversions",
  "start_date": "2025-06-01",
  "end_date": "2025-08-31"
}

// Standard Success Response (200/201)
{
  "id": 42,
  "name": "Summer Campaign 2025",
  "company_id": 7,
  "status": "draft",
  "created_at": "2025-04-02T10:30:00Z"
}

// Standard Error Response (400/401/403/404)
{
  "error": "Campaign not found or access denied",
  "statusCode": 404
}
```

---

14. Asset & File Storage Architecture

Dual Storage System

```
┌──────────────────────────────────────────────────────────────┐
│                    ASSET STORAGE DESIGN                       │
│                                                               │
│  Current State: Dual System (transitional)                    │
│                                                               │
│  ┌──────────────────────────┐  ┌──────────────────────────┐  │
│  │  FILE SYSTEM (legacy)    │  │  DATABASE (new)          │  │
│  │                          │  │                          │  │
│  │  /app/Assets/            │  │  asset_library table     │  │
│  │  /app/Assets Library/    │  │  (metadata + file_url)   │  │
│  │                          │  │                          │  │
│  │  • Ad images             │  │  • file_name             │  │
│  │  • Generated creatives   │  │  • file_url              │  │
│  │  • Uploaded media        │  │  • platform              │  │
│  │                          │  │  • aspect_ratio          │  │
│  │  Served via:             │  │  • type (image/video)    │  │
│  │  nginx /assets/* proxy   │  │  • company_id            │  │
│  │  nginx /library/* proxy  │  │  • created_at            │  │
│  └──────────────────────────┘  └──────────────────────────┘  │
│                                                               │
```

```
│  Docker Volumes (persistent):                                    │
│  • assets:/app/Assets                                            │
│  • library:/app/Assets Library                                   │
└──────────────────────────────────────────────────────────────────┘
```

---

15. AI Integration Architecture

Gemini AI Usage Map

```
┌──────────────────────────────────────────────────────────────────┐
│              GOOGLE GEMINI 2.0 FLASH INTEGRATION                 │
│                                                                  │
│  Config: Environment variable GEMINI_API_KEY                     │
│  SDK:    Google Generative AI SDK for .NET                       │
│                                                                  │
│  Use Cases:                                                      │
│  ┌──────────────────────────────────────────────────────────┐   │
│  │ Feature              │ Endpoint              │ Model      │   │
│  ├──────────────────────┼───────────────────────┼────────────┤   │
│  │ Ad Copy Generation   │ /api/ai/generate-copy │ Flash 2.0  │   │
│  │ Creative Brief       │ /api/ai/brief         │ Flash 2.0  │   │
│  │ Brand Compliance     │ /api/ai/brand-check   │ Flash 2.0  │   │
│  │ Performance Insights │ /api/ai/insights      │ Flash 2.0  │   │
│  │ A/B Test Analysis    │ /api/ai/ab-analysis   │ Flash 2.0  │   │
│  │ Image Prompts        │ /api/ai/image-prompt  │ Flash 2.0  │   │
│  └──────────────────────────────────────────────────────────┘   │
│                                                                  │
│  BrandComplianceService.cs:                                      │
│  → Loads brand_guidelines for company                            │
│  → Sends ad content + brand rules to Gemini                      │
│  → Returns compliance score + violation reasons                  │
└──────────────────────────────────────────────────────────────────┘
```

---

16. Service Layer Architecture

Backend Services

```
backend/Services/
│
├── DeploymentOrchestrator.cs
│   ┌──────────────────────────────────────────────────────────┐
│   │  Coordinates multi-platform deployment                    │
│   │                                                           │
│   │  Deploy(ad, platform, ad_account)                        │
│   │      │                                                    │
│   │      ├── "facebook"  → FacebookAdsService.CreateAd()      │
│   │      ├── "instagram" → FacebookAdsService.CreateAd()      │
│   │      ├── "tiktok"    → TikTok API (direct in Program.cs)  │
│   │      ├── "google"    → GoogleAdsService.CreateAd()        │
```

```
│   │      └── "youtube"   → YouTubeAdsService.CreateAd()         │
│   │                                                              │
│   │  Returns: { external_ad_id, platform_url, status }          │
│   └──────────────────────────────────────────────────────────────┘
│
├── FacebookAdsService.cs
│   ┌──────────────────────────────────────────────────────────────┐
│   │  Wraps Facebook Graph API v19.0                              │
│   │  • CreateCampaign(), CreateAdSet(), CreateAd()              │
│   │  • UpdateBudget(), PauseAd(), ResumeAd()                    │
│   │  • GetInsights() → pull impressions, clicks, spend          │
│   └──────────────────────────────────────────────────────────────┘
│
├── GoogleAdsService.cs
│   ┌──────────────────────────────────────────────────────────────┐
│   │  Wraps Google Ads API                                        │
│   │  • CreateCampaign(), CreateAdGroup(), CreateAd()            │
│   │  • GetMetrics() → pull CPC, CTR, conversions                │
│   └──────────────────────────────────────────────────────────────┘
│
├── YouTubeAdsService.cs
│   ┌──────────────────────────────────────────────────────────────┐
│   │  Wraps YouTube Data API                                      │
│   │  • CreateVideoAd(), GetVideoMetrics()                       │
│   └──────────────────────────────────────────────────────────────┘
│
├── MetricsFetchService.cs
│   ┌──────────────────────────────────────────────────────────────┐
│   │  Aggregates metrics from all platforms                       │
│   │  • FetchAll(company_id) → pulls from all connected accounts  │
│   │  • Stores results in ad_performance table                    │
│   │  • Called on-demand or scheduled                            │
│   └──────────────────────────────────────────────────────────────┘
│
└── BrandComplianceService.cs
    ┌──────────────────────────────────────────────────────────────┐
    │  AI-powered brand guideline enforcement                      │
    │  • LoadGuidelines(company_id) → from brand_guidelines table  │
    │  • CheckCompliance(adContent, guidelines) → Gemini API       │
    │  • Returns: { isCompliant, score, violations[] }            │
    └──────────────────────────────────────────────────────────────┘
```

---

17. Data Flow Diagrams

Campaign Creation Flow

```
Expert                 Backend                  Database           Gemini AI
  │                      │                        │                   │
  ├──POST /api/campaigns──►│                       │                   │
  │                        ├──INSERT campaigns──────►│                  │
  │  {id: 1, status:draft} │◄── campaign record ────┤                  │
```

```
  │◄──────────────────────┤                       │                   │
  │                       │                       │                   │
  ├──POST /api/ad-sets────►│                       │                   │
  │                       ├──INSERT ad_sets──────►│                   │
  │◄──────────────────────┤                       │                   │
  │                       │                       │                   │
  ├──POST /api/ads────────►│                       │                   │
  │                       ├──INSERT ads──────────►│                   │
  │◄──────────────────────┤                       │                   │
  │                       │                       │                   │
  ├──POST /api/ai/generate─►│                       │                   │
  │                       ├────── Gemini prompt ──────────────────►│
  │                       │◄────── generated text ──────────────────┤
  │                       ├──INSERT asset_library──►│                   │
  │  {asset_url, content}  │                       │                   │
  │◄──────────────────────┤                       │                   │
```

Analytics Fetch Flow

```
Expert/CMO              Backend               Platform APIs          Database
    │                     │                       │                     │
    ├─GET /api/analytics──►│                       │                     │
    │                     │                       │                     │
    │                     ├─MetricsFetchService   │                     │
    │                     │   │                   │                     │
    │                     │   ├─Facebook Insights─►│                     │
    │                     │   │◄──────────────────┤                     │
    │                     │   ├─TikTok Analytics──►│                     │
    │                     │   │◄──────────────────┤                     │
    │                     │   ├─Google Ads Metrics►│                     │
    │                     │   │◄──────────────────┤                     │
    │                     │                       │                     │
    │                     ├──INSERT ad_performance──────────────────────►│
    │                     │                                             │
    │  { metrics[] }      │◄──────── aggregated metrics ────────────────┤
    │◄────────────────────┤                                             │
```

---

18. Security Architecture

Security Layers

```
┌──────────────────────────────────────────────────────────────────────┐
│                      SECURITY ARCHITECTURE                           │
│                                                                      │
│  Layer 1: Transport Security                                         │
│  ────────────────────────────                                        │
│  • HTTPS in production (nginx TLS termination)                       │
│  • HTTP internally between containers (private Docker network)       │
│                                                                      │
│  Layer 2: Authentication                                             │
```

```
│  ────────────────────────                                          │
│  • JWT HS256 tokens, 8-hour expiry                                  │
│  • BCrypt password hashing (industry standard)                      │
│  • No session state — stateless API                                 │
│                                                                     │
│  Layer 3: Authorization                                             │
│  ────────────────────────                                          │
│  • Every API endpoint checks JWT                                    │
│  • company_id extracted from JWT — no client-supplied tenant ID     │
│  • Role extracted from JWT — server-side role enforcement           │
│  • [Authorize] attribute on all protected routes                    │
│                                                                     │
│  Layer 4: Data Isolation                                            │
│  ────────────────────────                                          │
│  • Every query filters by company_id from JWT claim                 │
│  • Impossible to access another company's data via API             │
│  • Super Admin is the only cross-tenant role                        │
│                                                                     │
│  Layer 5: Secret Management                                         │
│  ──────────────────────────                                        │
│  • Secrets via environment variables (Docker env / .env file)       │
│  • appsettings.example.json has no real secrets                     │
│  • .gitignore excludes appsettings.json, .env files                 │
│  • Gemini API key: ${GEMINI_API_KEY} from host environment          │
└─────────────────────────────────────────────────────────────────────┘
```

---

19. Scalability & Performance Design

Current Architecture Constraints & Scaling Path

```
┌─────────────────────────────────────────────────────────────────────┐
│                  SCALABILITY CONSIDERATIONS                         |
│                                                                     │
│  Current State (Single Server):                                     │
│  ┌──────────────────────────────────────────────────────────┐   |
│  │  1 Docker host → 3 containers (nginx, backend, postgres)    |   |
│  │  Suitable for: MVP, small-medium load                       |   |
│  └──────────────────────────────────────────────────────────┘   |
│                                                                     │
│  Horizontal Scaling Path:                                           │
│  ┌──────────────────────────────────────────────────────────┐   |
│  │                                                             |   |
│  │  Load Balancer (nginx / cloud LB)                          |   |
│  │       │                                                     |   |
│  │       ├── Backend Instance 1  ┐                            |   |
│  │       ├── Backend Instance 2  ├── Shared PostgreSQL        |   |
│  │       └── Backend Instance N  ┘   (or RDS/Cloud DB)       |   |
│  │                                                             |   |
│  └──────────────────────────────────────────────────────────┘   |
│                                                                     │
│  Performance Optimizations in Place:                                │
```

```
│  • PostgreSQL indexes on all FK + high-query columns              │
│  • Stateless JWT → no session affinity needed                     │
│  • Static assets served by nginx (not backend)                    │
│  • EF Core compiled queries for hot paths                         │
│                                                                   │
│  Future Optimizations:                                            │
│  • Redis cache layer for analytics + AI responses                 │
│  • Background job queue (Hangfire) for AI generation              │
│  • S3/Blob storage for assets (replacing file system)             │
│  • Read replicas for analytics queries                            │
└──────────────────────────────────────────────────────────────────┘
```

---

20. Architecture Summary

Component Count

Layer	Technology	Count/Detail
Frontend Screens	Vanilla JS SPA	36 screens
API Endpoints	ASP.NET Core 9 Minimal API	95+ endpoints
Database Tables	PostgreSQL 16	29 tables
Service Classes	C# Service Layer	6 services
Docker Containers	Docker Compose	3 containers
External Platforms	Ad Platform APIs	4 platforms
AI Engine	Google Gemini 2.0 Flash	1 AI provider
User Roles	RBAC System	5 roles

Technology Decision Summary

Decision	Choice	Reason
Frontend Framework	Vanilla JS (No Framework)	Lightweight, no dependency overhead
Build Tool	Vite 7.x	Fast HMR, modern bundling
CSS	Tailwind 4.x	Rapid UI without custom CSS
Backend	ASP.NET Core 9 Minimal API	High performance, minimal boilerplate
ORM	EF Core 9	Type-safe DB access, auto-schema
Database	PostgreSQL 16	ACID, JSON support, performance
Auth	JWT HS256	Stateless, scalable
AI	Google Gemini 2.0 Flash	Fast, cost-effective, multimodal
Containerization	Docker Compose	Portable, reproducible deployments

Reverse Proxy	Nginx	SPA routing + API proxy + static files

Document generated for AI-Marketing-New platform — all technical details sourced from actual implementation files.